

RAI-853: Advanced Robotics with ROS

Lec 04: Robot Modelling and Visualization

Instructor:

Dr. Khawaja Fahad Iqbal

Deputy HoD (Department of Robotics & AI, SMME, NUST)

Scientific Director (Intelligent Robotics Lab, NCAI)



- **Workspace**
- **Visualizing a Robot in RViz**
- **Creating a URDF**
- **URDF for a Mobile Robot**

- **Workspace**
- Visualizing a Robot in RViz
- Creating a URDF
- URDF for a Mobile Robot

- Before you create a node by yourself, there is a bit of setup to do. We need to create a **ROS 2 workspace**, in which we will build our application.
- In this workspace, we will then add packages to better organize our nodes.
- Then, in those packages, we can start to write our nodes. After we write a node, we will build it and run it.

- Nodes will exist within packages, and all our packages will exist within a ROS 2 workspace.
- A workspace is nothing more than a **folder organization** in which we will create and build our packages. Our entire ROS 2 application will live within this workspace.
- To create one, we need to follow certain rules. Let's create your first workspace step by step and correctly set it up.

- Open a terminal, navigate to your home directory, and create the workspace:

```
cd
```

```
mkdir ros2_ws
```

- Then, enter the workspace and create a new directory named src. This is where you will write all the code for your ROS 2 application:

```
cd ros2_ws/
```

```
mkdir src
```

- That's really all there is to it. To set up a new workspace, you just create a new directory (somewhere in your home directory) and create an src directory inside it.

- Even if the workspace is empty (we have not created any packages yet), we can still build it. To do that, follow these steps:

```
cd ~/ros2_ws/
```

```
colcon build
```

```
Summary: 0 packages finished [0.73s]
```

- As you can see, no packages were built, but let's list all directories under ~/ros2_ws:

```
ls
```

```
build install log src
```

- The **build** directory will contain the intermediate files required for the overall build.
- In **log**, you will find logs for each build.
- The most important directory for you is **install**, which is where all your nodes will be installed after you build the workspace.

- If you navigate inside the newly created **install directory**, you can see a `setup.bash` file:

```
cd install/
```

```
ls
```

```
COLCON_IGNORE _local_setup_util_ps1.py setup.ps1
```

```
local_setup.bash _local_setup_util_sh.py setup.sh
```

```
local_setup.ps1 local_setup.zsh setup.zsh
```

```
local_setup.sh setup.bash
```

- This might look familiar. If you remember, after we installed ROS 2, we sourced a **similar bash script** from the ROS 2 installation directory (`/opt/ros/jazzy/setup.bash`) so that we could use ROS 2 in our environment. We will need to do the same for our workspace.

- **Every time you build your workspace, you have to source it** so that the environment (the session you are in) knows about the new changes in the workspace.
- To source the workspace, source this setup.bash script:

```
source ~/ros2_ws/install/setup.bash
```
- Then, as we previously did, **we are going to add that line into our .bashrc**. This way, you don't need to source the workspace every time you open a new terminal.

- Open your `.bashrc` (located in your home directory the path is `~/.bashrc`) using any text editor you want:

`gedit ~/.bashrc`

- Add the line to source the workspace's `setup.bash` script, just after the one to source the global ROS 2 installation. The order is very important here. **You have to source the global ROS 2 installation first**, and then your workspace, not the other way around:

`source /opt/ros/jazzy/setup.bash`

`source ~/ros2_ws/install/setup.bash`

- Make sure to save `.bashrc`. Now, both ROS 2 and your workspace will be sourced in any new terminal you open.

- Workspace
- **Visualizing a Robot in RViz**
- Creating a URDF
- URDF for a Mobile Robot

- RViz allows you to visualize a robot model in 3D and contains many plugins and functionalities that will help you develop your robotics applications.
- As we haven't created a robot model yet, we will use one from an existing ROS 2 package named `urdf_tutorial`.
- We will load a robot model in RViz and learn how to navigate the software.

- There is no need to install RViz. It was already included when you installed ROS 2.
- To visualize a robot model on RViz, we will install a new ROS package named **urdf_tutorial**. This package contains some existing launch files and robot model files (how to create a robot model will be the focus of the next chapter).
- Open a terminal and install this package:
sudo apt install ros-<distro>-urdf-tutorial
- Then, so that you can use the package, make sure you source the environment.

- The urdf_tutorial package contains a **launch file, named display.launch.py**, that will start RViz and load a robot model into it.
- Navigate to the share directory where the package was installed, and you will find a urdf folder under the package name:

```
cd /opt/ros/<distro>/share/urdf_tutorial/urdf/
```

- In the urdf folder, you can find several robot model files:

```
ls
```

01-myfirst.urdf

04-materials.urdf

07-physics.urdf

02-multipleshapes.urdf

05-visual.urdf

08-macroed.urdf.xacro

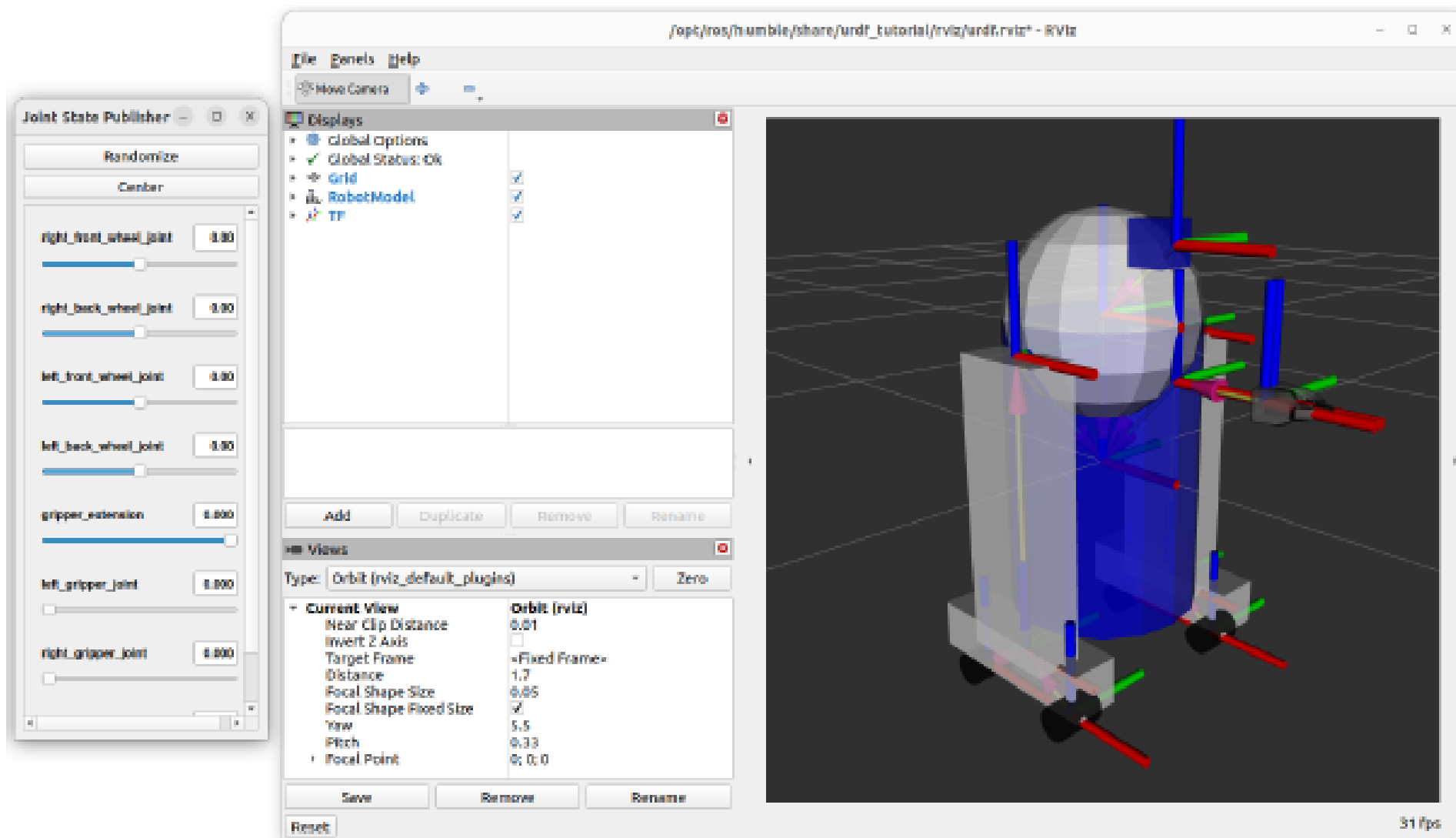
03-origins.urdf

06-flexible.urdf

- You can now start a robot model in RViz by launching the `display.launch.py` file and add the path to the robot model with an additional model argument after the launch file:

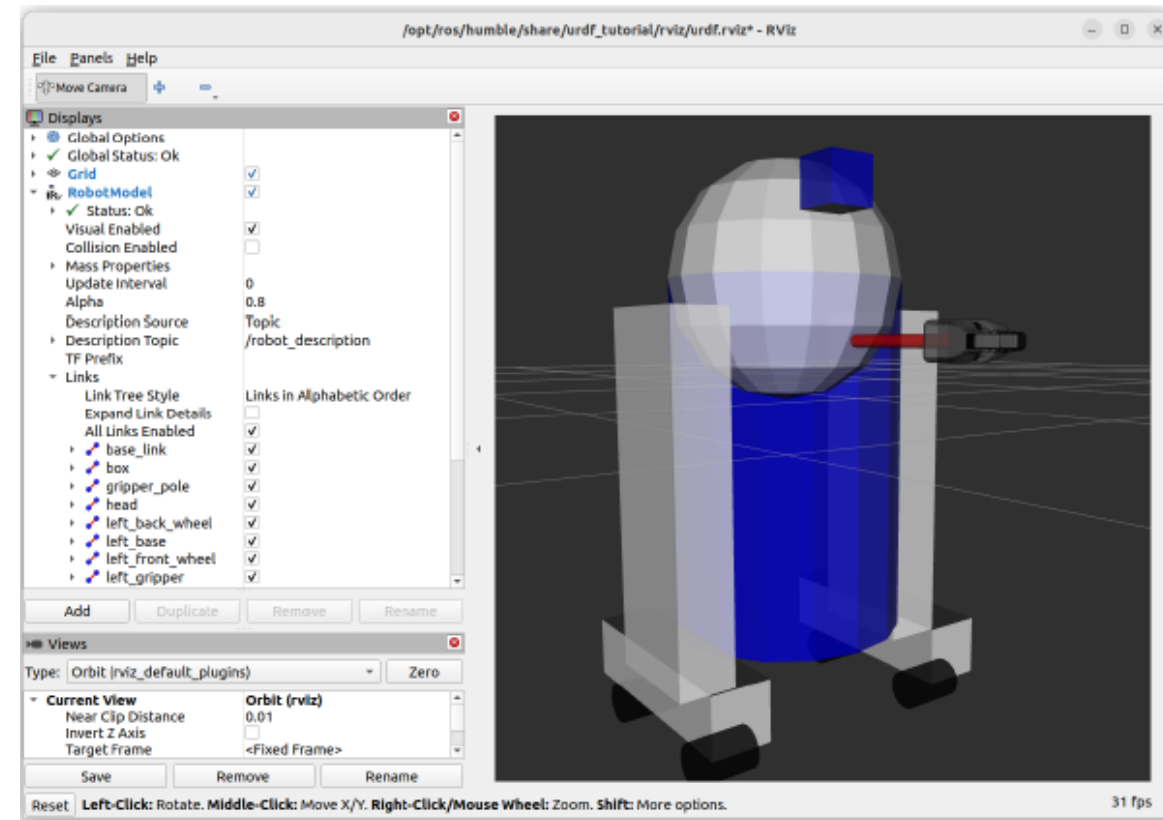
```
ros2 launch urdf_tutorial display.launch.py model:=/opt/  
ros/<distro>/share/urdf_tutorial/urdf/07-physics.urdf
```

- After running the command, you should see something like this:

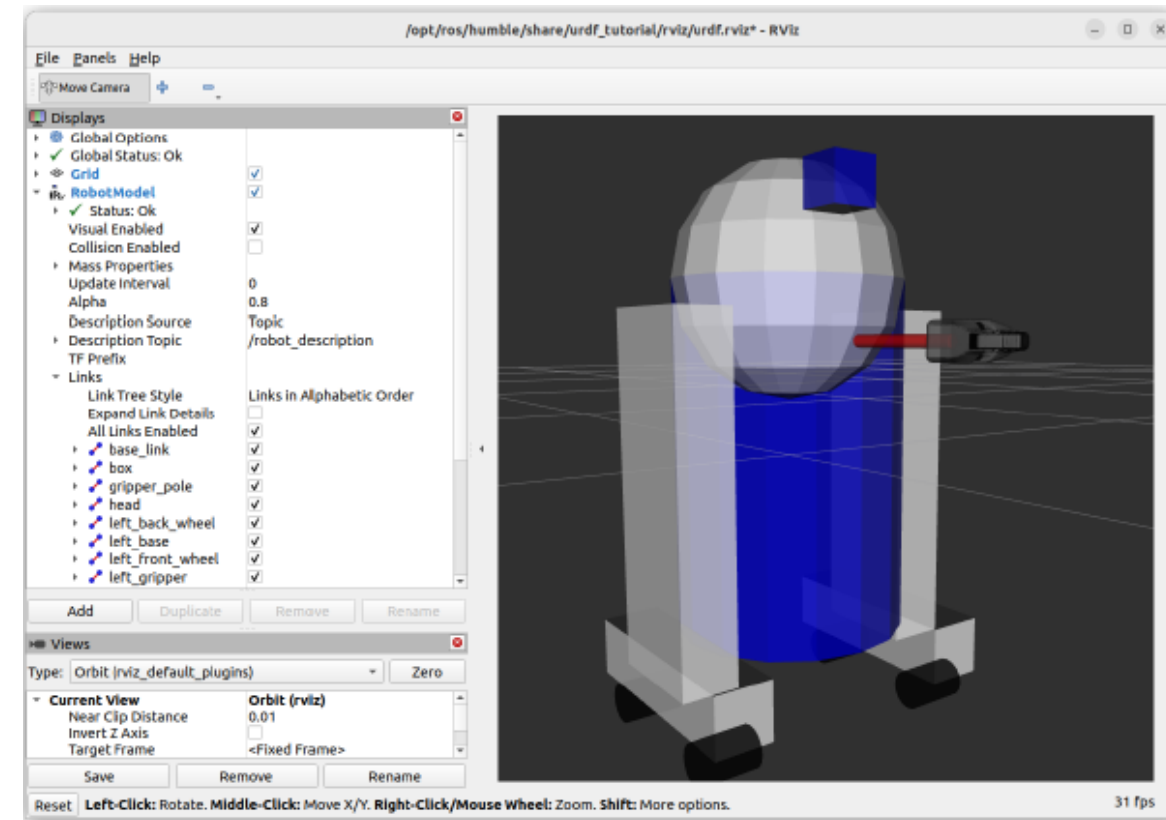


- You will get two windows: the main one (**RViz**) with the robot model, and a **Joint State Publisher** window with some cursors.
- The robot model we have here is a replica of a famous science-fiction movie robot. It has some wheels, a torso, a head, and a gripper.
- Let's focus on the main window (**RViz**) for now. Take some time to learn how to navigate in the 3D space and move around the robot. You can use the left click, right click, and mouse wheel. For this, it's best to have a mouse, but you could still manage to navigate with the touchpad of a laptop, although it's less ergonomic.

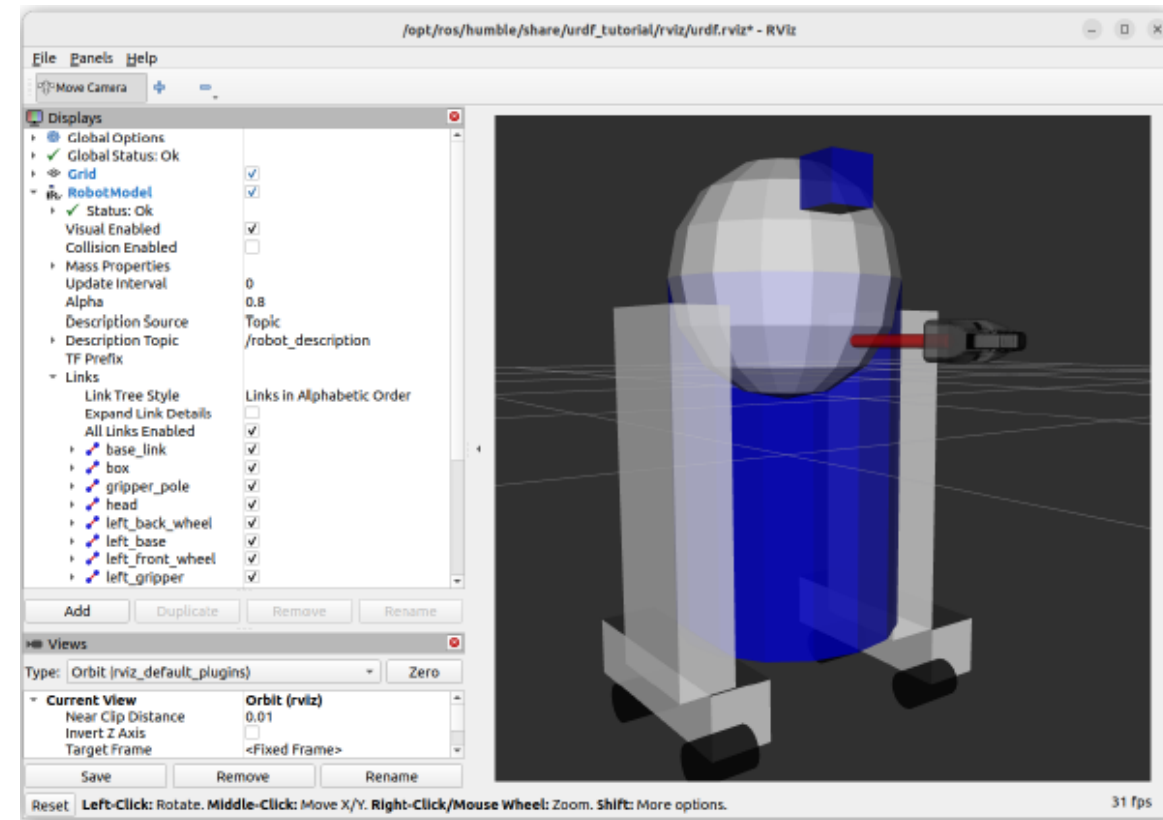
- Look at the menu on the left side of the **RViz** window. There, you will see, in blue bold letters, **RobotModel** and **TF**. As you can see, you can enable or disable both menus.
- Disable **TF**, keep **RobotModel**, and expand the menu. There, you can find a submenu called **Links**.



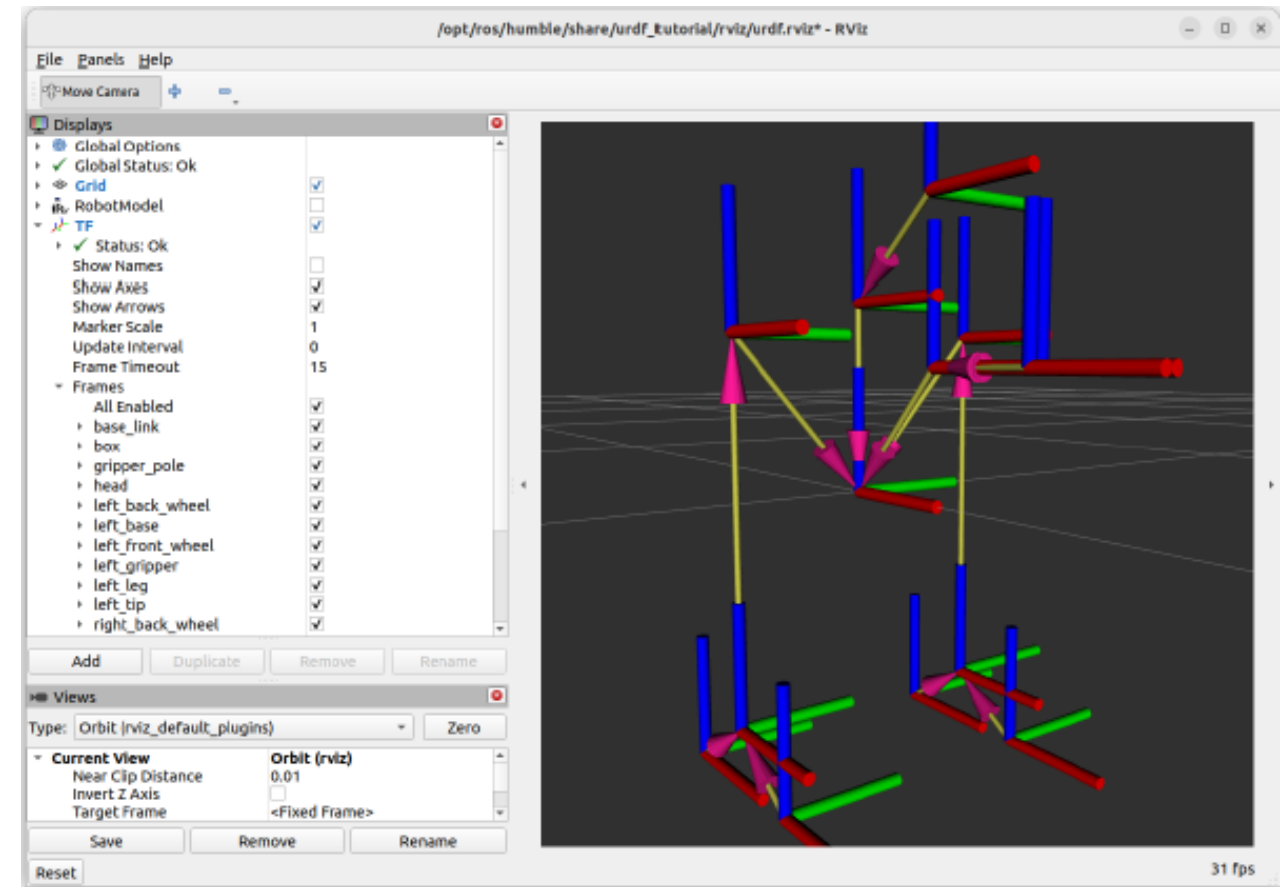
- Check and uncheck some boxes.
- As you can see from this menu, a *link* is one rigid part (meaning one solid part with no articulation) of the robot.
- Basically, in ROS, a robot model will consist of a collection of rigid parts put together.



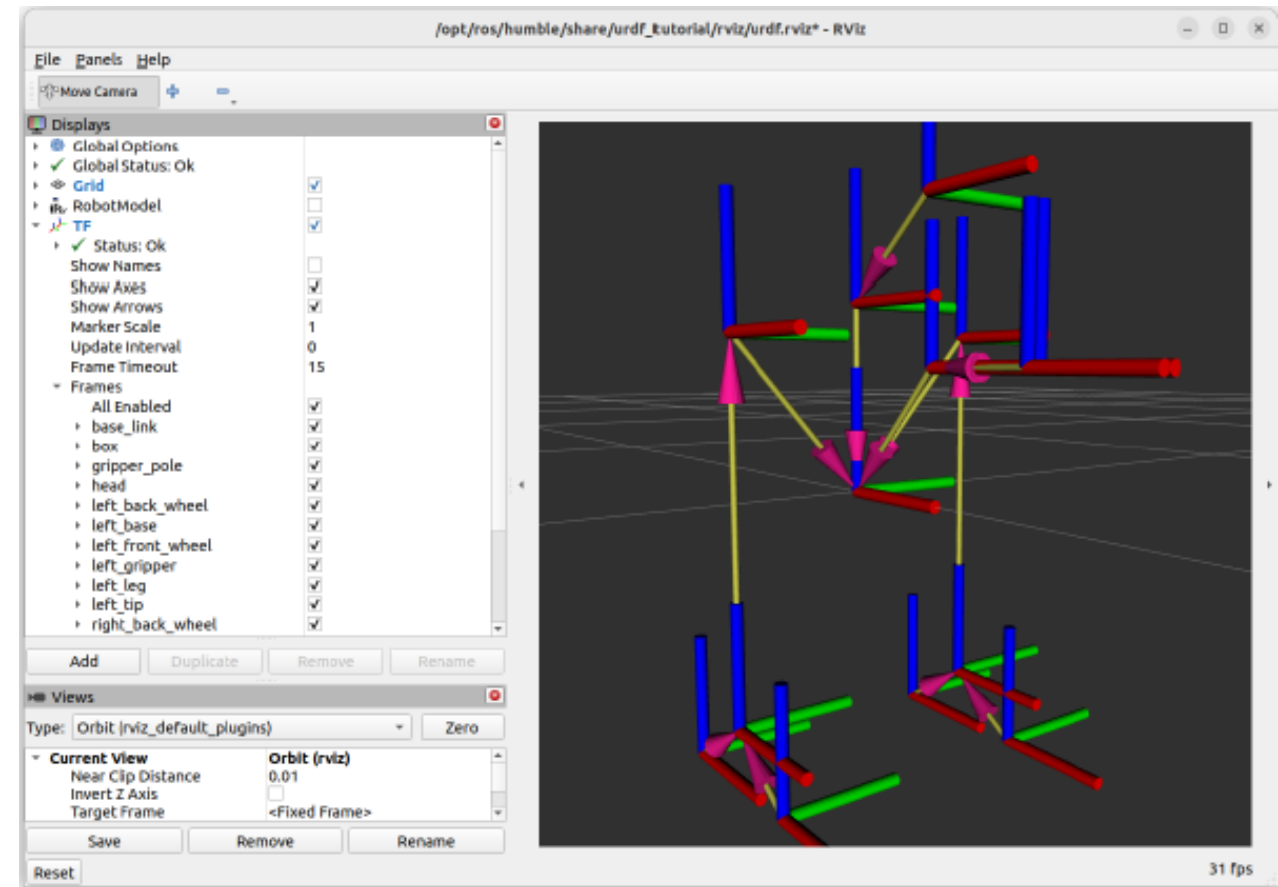
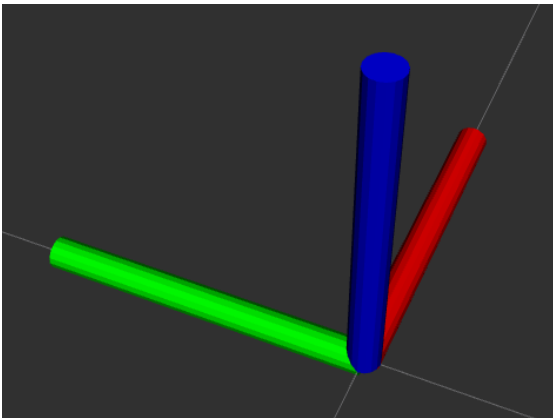
- In this example, links are represented by basic shapes: **boxes, cylinders, and spheres.**
- Those rigid parts do nothing on their own, so how are they connected, and how do they move between each other?
- This is the job of TFs.



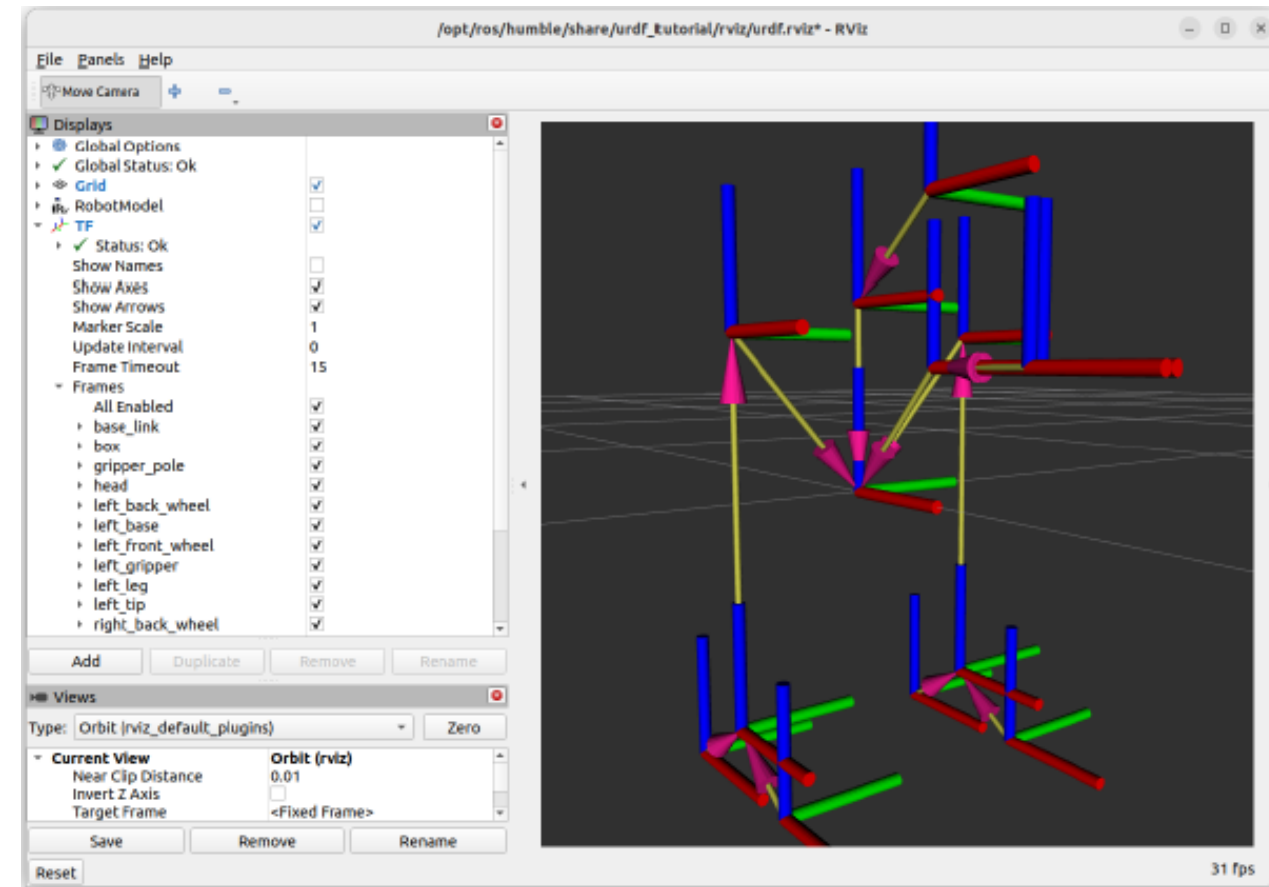
- Let's now check the **TF** box. You can keep **RobotModel** checked or unchecked.
- Inside the **TF** menu, you have a submenu called **Frames**, and you can also enable or disable each frame for the robot.
- The **axes** you see here (red, green, and blue coordinate systems) represent the frames, or basically the **origin** of each link of the robot.



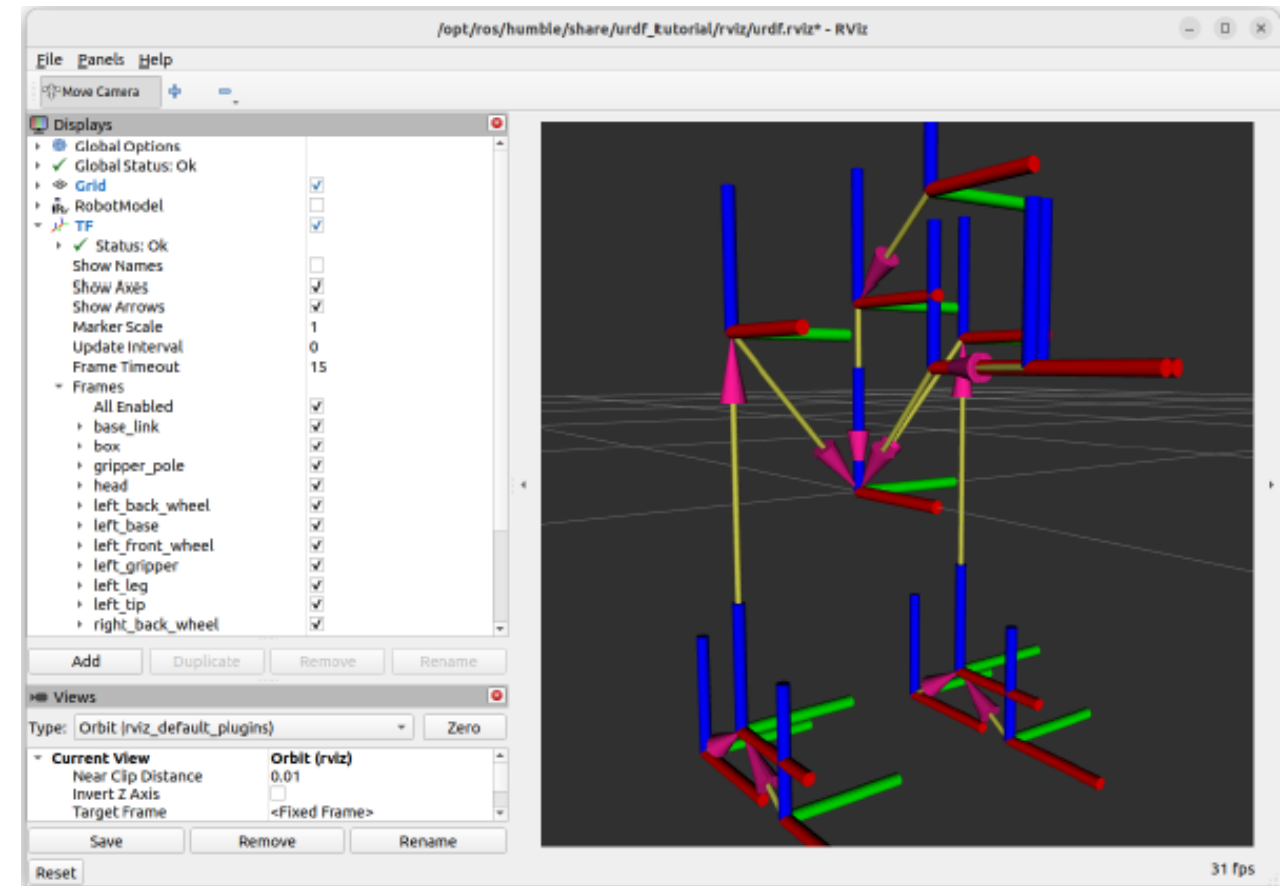
- Coordinate systems follow the right-hand rule in ROS.
 - X axis (red) pointing forward
 - Y axis (green) pointing 90 degrees to the left
 - Z axis (blue) pointing up



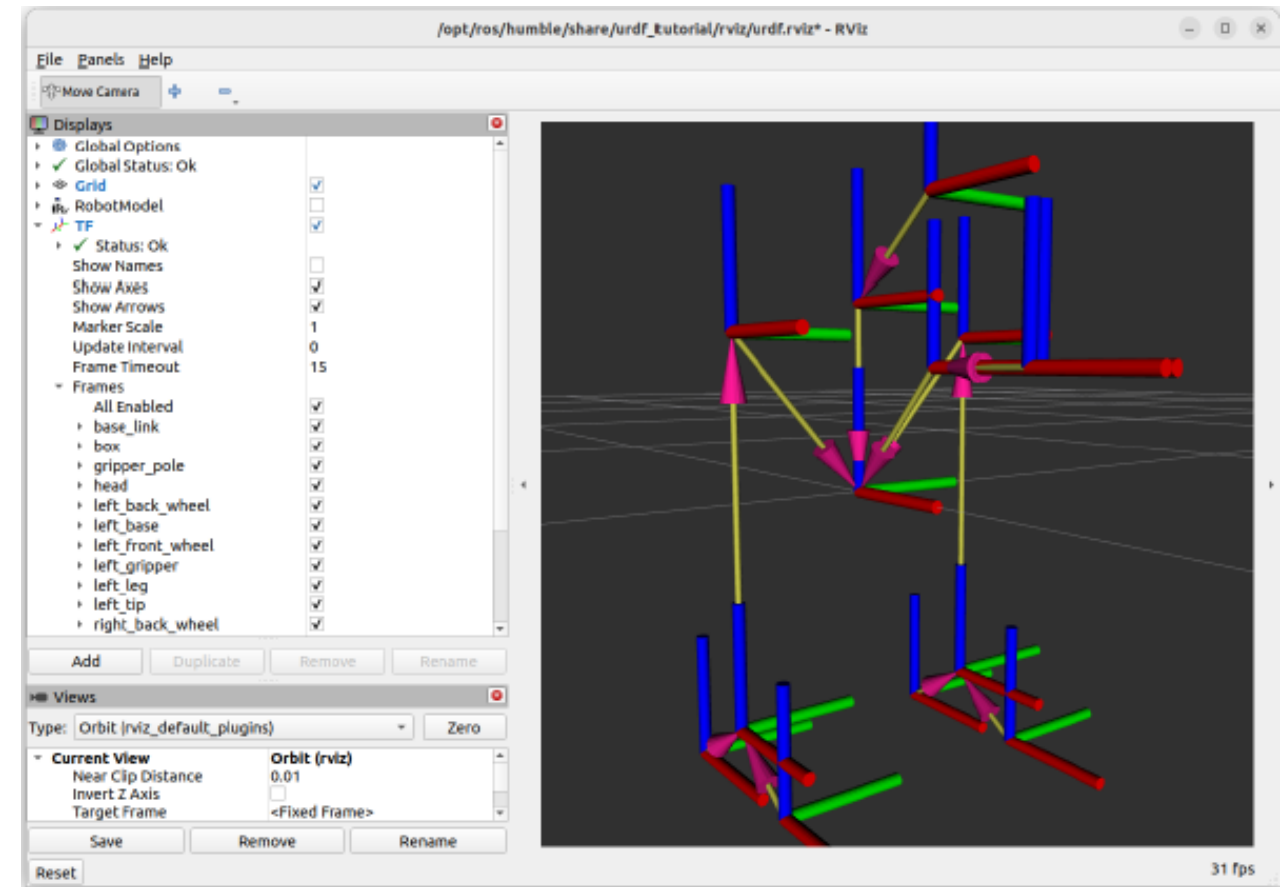
- Each rigid part will be linked to another rigid part, thanks to a **TF**.
- This transformation defines how those two parts are placed relative to each other.
- In addition to that, the TF also defines whether the two parts are moving, and if so, how—translation, rotation, and so on.



- The arrows that you see between each frame are the relationship between each rigid part (link) of the robot. A **TF** is represented by an arrow.
- To sum up:
 - **Link:** one rigid part of a robot
 - **Frame:** the origin of a link (axis in RViz)
 - **TF:** the relationship between two frames (arrow in RViz)



- To make some parts of the robot move, you can move some of the cursors in the **Joint State Publisher** window.
- You will see the frames and TFs moving in RViz.
- If you also check the **RobotModel** box again, you will see the rigid parts moving as well.



- Let's list all nodes:

ros2 node list

/joint_state_publisher

/robot_state_publisher

/rviz

/transform_listener_impl_5a530d0a8740

- You can see that RViz is actually started as a node (rviz). We also have the joint_state_publisher and robot_state_publisher nodes

- Now, let's list all topics:

ros2 topic list

/joint_states

/parameter_events

/robot_description

/rosout

/tf

/tf_static

- In this topic list, we find **the /tf topic**.
- This topic will be there for any robot you create.
- The TFs you saw on RViz are actually the 3D visualization of this topic—meaning **the rviz node is a subscriber to the /tf topic**.
- You can subscribe to the topic from the terminal with the following:

ros2 topic echo /tf

- If you do so, you will receive lots of messages. Here is an extract:

transforms:

- header:

stamp:

sec: 1719581158

nanosec: 318170246

frame_id: base_link

child_frame_id: gripper_pole

transform:

translation:

x: 0.19

y: 0.0

z: 0.2

rotation:

x: 0.0

y: 0.0

z: 0.0

w: 1.0

- Workspace
- Visualizing a Robot in RViz
- **Creating a URDF**
- URDF for a Mobile Robot

- Now you will dive directly in and create your first URDF.
- We will first create an **XML file** for the URDF. In this file, we will add a link, which will represent one rigid part of a robot, and visualize it in RViz.
- We will also explore the different types of shapes you can use—boxes, cylinders, and so on.

- Open a terminal and create a new file in your home directory:

```
cd
```

```
touch my_robot.urdf
```

- You can then open this file with any text editor or IDE, for example, with VS Code:

```
code my_robot.urdf
```

- Here is the minimum code you have to write inside a URDF file:

```
<?xml version="1.0"?>
```

```
<robot name="my_robot">
```

```
</robot>
```

- To create a visual element for a link, you can use existing shapes: **boxes, cylinders, and spheres.**
- We can also include a custom shape made from **Computer-Aided Design (CAD)** software.
- To get started, let's imagine the main base of a robot, represented as a box. The box is 60 cm x 40 cm x 20 cm, or 0.6 m x 0.4 m x 0.2 m.

- Here is the code for this first link:

```
<robot name="my_robot">  
  <link name="base_link">  
    <visual>  
      <geometry>  
        <box size="0.6 0.4 0.2" />  
      </geometry>  
      <origin xyz="0 0 0" rpy="0 0 0" />  
    </visual>  
  </link>  
</robot>
```

- The **<link> tag** defines the link. All the properties for this link must be inside the tag. You also have to provide a name attribute for the link. As a convention, for the first link, we use `base_link`.
- Then, inside this tag, we have the **<visual> tag**. If you want to define a visual appearance for the link (rigid part), you can do so with this tag. Inside, you will have the following:

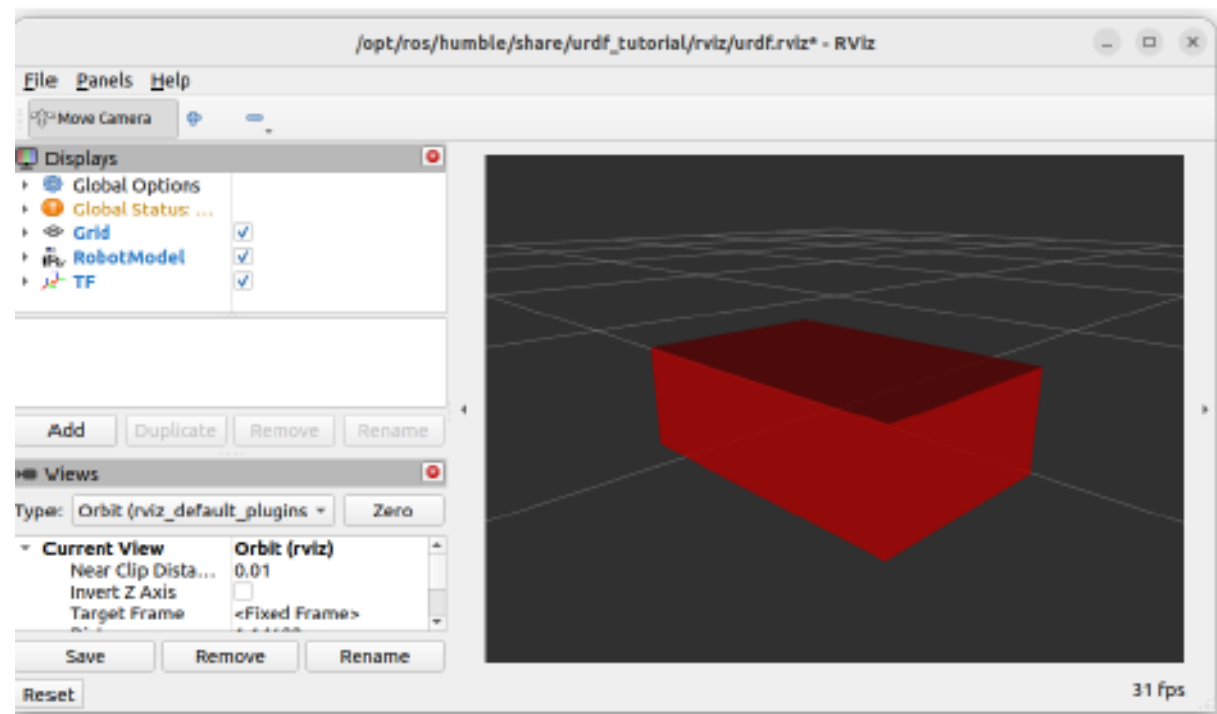
- **<geometry>**: This will define the shape of the link. Here, we use the **<box>** tag and provide the dimensions with the size attribute.
- **<origin>**: This tag is quite important, as it defines the origin of the visual relative to the origin of the link. The origin contains six elements for translation and rotation.

- Now we can visualize the URDF in RViz. The goal here is to see the box in 3D and verify that everything is correct.
- This can be done by following these steps:
 1. Make the smallest modification (add or modify something).
 2. Visualize the URDF in Rviz.
 3. If correct, continue with the next feature; if not, go back, fix it, and check again.

- Open a terminal and start the `display.launch.py` launch file, with the absolute path to your URDF file for the model argument:

```
ros2 launch urdf_tutorial
display.launch.py \
model:=/home/<user>/my_robot.urdf
```

- You will then see a box (red color by default) inside **RViz**. You will also have the **Joint State Publisher** window, empty, with no cursor.



- Now that you have a URDF file with one link, let's add another link, and connect them with a joint. This joint will be used to generate a TF.
- Properly assembling two links with a joint is the main problem anybody faces when learning URDF.
- There are several origins and axes you can modify, and getting two parts to be correctly placed between each other, with the correct movement, can be challenging.

- For this example, we want to add a cylinder (radius: 0.1 m, length: 0.3 m) on top of the box.
- We will make this cylinder gray (to create a contrast with the green box), so, let's first create another `<material>` tag with gray color.
- You can add this new tag just after the robot name in the urdf:

```
<material name="gray">
```

```
  <color rgba="0.7 0.7 0.7 1" />
```

```
</material>
```

- Create another `<link>` tag inside the `<robot>` tag, with the specifications for the second link. You can place this link just after the `base_link`:

```
<link name="shoulder_link">
```

```
  <visual>
```

```
    <geometry>
```

```
      <cylinder radius="0.1" length="0.3" />
```

```
    </geometry>
```

```
    <origin xyz="0 0 0" rpy="0 0 0" />
```

```
    <material name="gray" />
```

```
  </visual>
```

```
</link>
```

- There are three types of basic shapes you can use. For each one you will need to provide the dimensions with different attributes:
- **<box>**: You need to add a size argument with three components: x, y, and z
- **<cylinder>**: You need to add two arguments, radius and length
- **<sphere>**: You need only one argument, radius

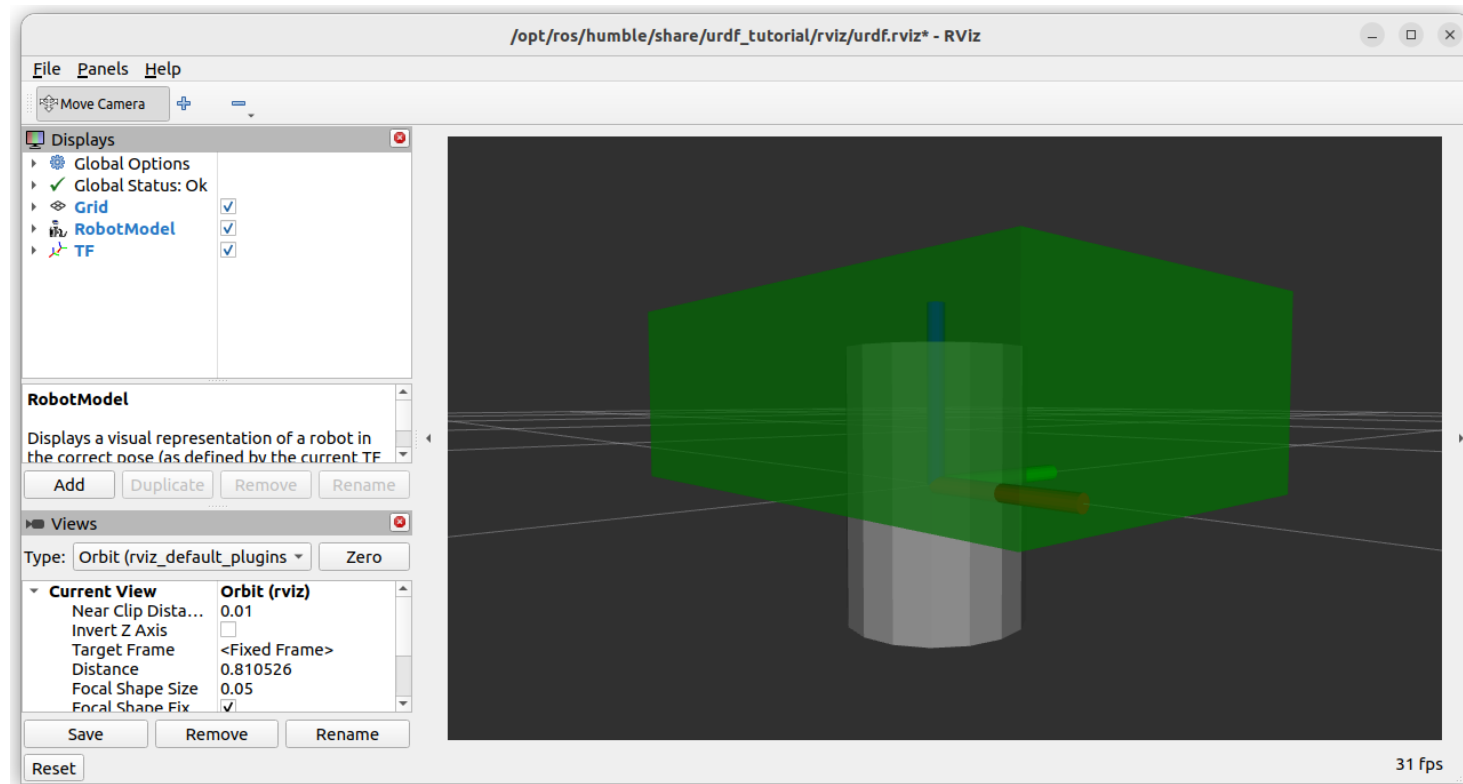
- To define **how two links are connected**, you need to add a joint. Here is the code you can write after the two `<link>` tags (and still inside the `<robot>` tag):

```
<joint name="base_shoulder_joint" type="fixed">  
  <parent link="base_link" />  
  <child link="shoulder_link" />  
  <origin xyz="0 0 0" rpy="0 0 0" />  
</joint>
```

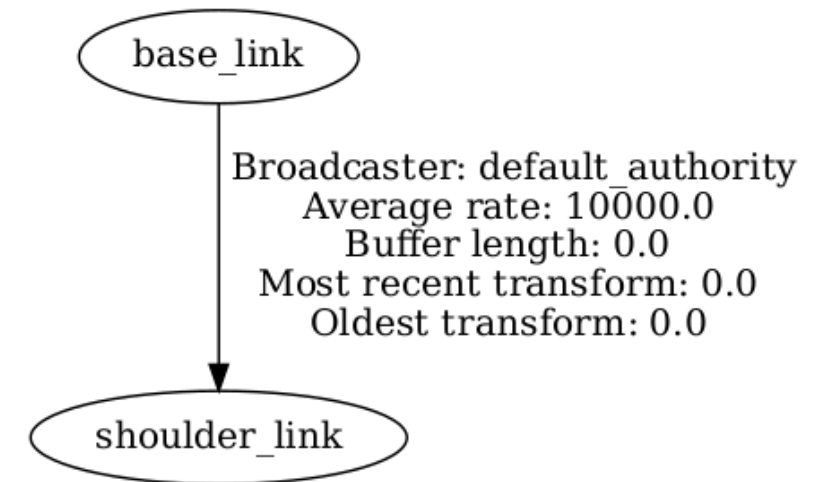
- To create a joint, you add a `<joint>` tag, which contains two attributes:
- **name:** You can choose whatever you want, as long as it makes sense. I usually combine the names of the two links I want to connect: `base_link` and `shoulder_link` become `base_shoulder_joint`.
- **type:** We will come back to the different joint types you can use in *Step 4*. For now, we set it as `fixed`, which means that the two links won't move between each other.

- Inside the <joint> tag, you then have three more tags:
- **<parent>**: You have to write the exact name of the parent link with the link attribute.
- **<child>**: You will write the exact name of the child link with the link attribute.
- **<origin>**: This will define the **origin of the child link relative to the origin of the parent link**. Once again, we use xyz for translation and rpy for rotation.

- With this code, you can start RViz again, and as there is a relationship between the links, the URDF will be displayed. Here is what you will get:



- As you can see, we now have a box (base_link) and a cylinder (shoulder_link). As all origin elements are set to 0, both link origins are at the same place.
- On top of that, you can also validate that the TF you've created with the joint is correctly placed in the TF tree.
- With the **ros2 run tf2_tools view_frames** command, you can generate the TF tree. In the newly created PDF file, you will see this:



- This is the step where most people get confused. If you look at the current code, we have three <origin> tags: one in each link and one in the joint. So, **which origin do we need to modify?**
- The classic error is to start to modify several random origins at once and try to find something that works by tinkering with the values.
- Even if that seems to work in the end, it will probably create more problems when you add other joints.

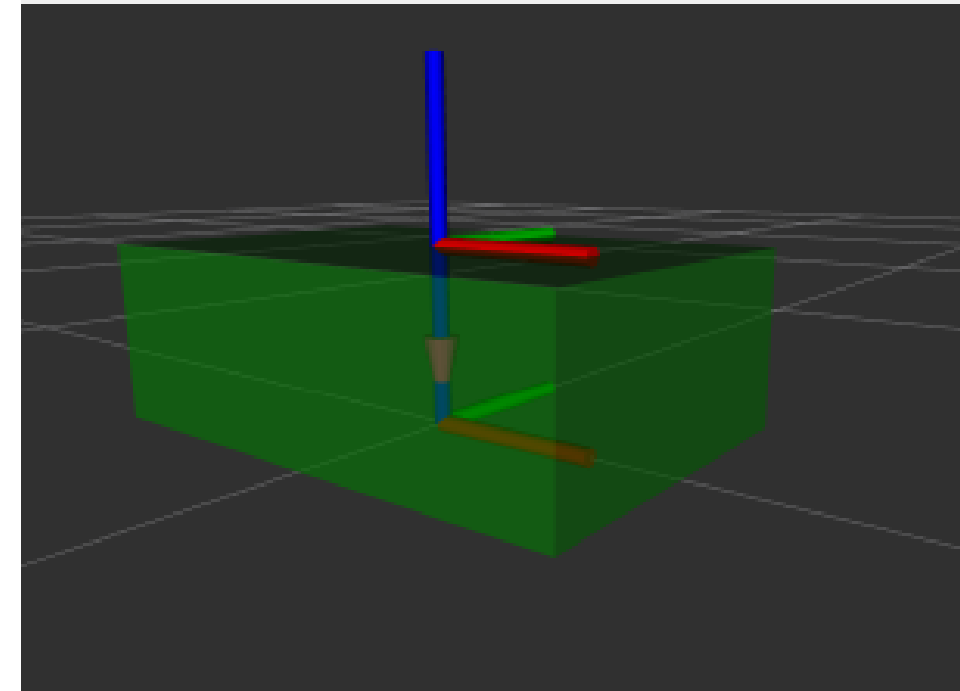
- So, for this step, you have to follow the *exact* process every time.
- **The first thing to do is to modify the <origin> tag of the joint**, so you get the frame of the child link correctly placed.
- This is what matters the most. You will first fix the joint origin, and then, and *only* then, fix the visual origin.

- For this, expand **RobotModel** and **Links**, and uncheck the `shoulder_link` visual (at this stage, seeing the visual can cause confusion, so we disable it).
- Then, ask yourself this question: where should be the frame for the `shoulder_link`, relative to the frame of the `base_link`?
- We want the `shoulder_link` to be on top of the box, so we need to move the frame by the height of the box; here, that's 0.2 m. There is no rotation needed for this joint, just a translation.

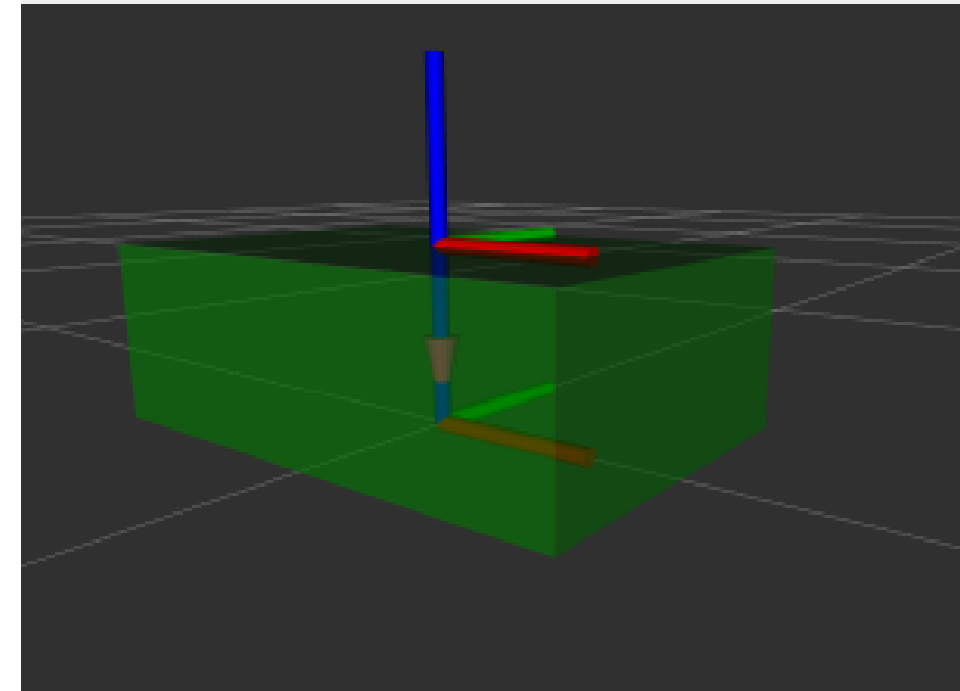
- So, you can now modify the <origin> tag inside the <joint> tag:

```
<origin xyz="0 0 0.2" rpy="0 0 0" />
```

- Start RViz again, disable the visual for the shoulder_link, and check whether the frame is correctly placed.
- Great, it seems that the frame for the shoulder_link is in the right place: on top of the box.



- This origin is the most important; this is what will define the TF. In the end, **TFs will be generated with all the joint origins in your URDF.**
- The visual will not be taken into account (**visuals will be used for inertial and collision** properties in the Gazebo simulator).
- We have our joint origin correctly set up. Now, let's see how to specify the joint type, so we can define the movement between the two links.



- The most common movements you will find in any robot will be as follows:
- **Fixed:** As previously mentioned, this is if you have two parts that are not moving
- **Revolute:** A rotation with a minimum and maximum angle, for example, in robotic arms
- **Continuous:** An infinite rotation, usually used for wheels
- **Prismatic:** If you need to make a part of your robot slide (only translation, no rotation)

- Let's say that the shoulder link is rotating (with a minimum and maximum limit) on the Z-axis on top of the box.
- You can specify this with the **revolute joint type**.
- By doing this, you will also need to add an <axis> tag to specify the axis of rotation, and a <limit> tag for the minimum and maximum angle

Step 4 – Setting Up the Joint Type

- Let's modify the `<base_shoulder_joint>` tag:

```
<joint name="base_shoulder_joint" type="revolute">
```

```
  <parent link="base_link" />
```

```
  <child link="shoulder_link" />
```

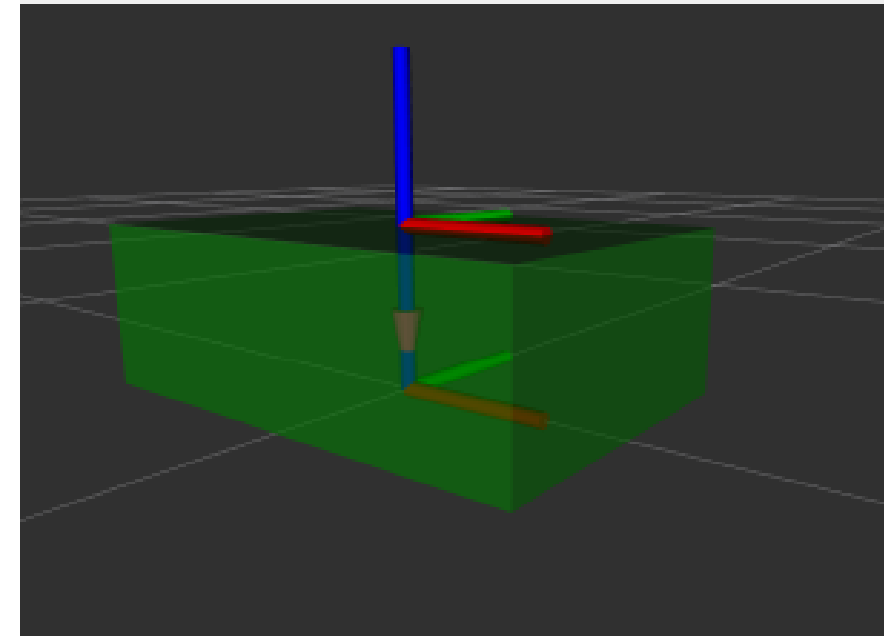
```
  <origin xyz="0 0 0.2" rpy="0 0 0" />
```

```
  <axis xyz="0 0 1" />
```

```
  <limit lower="-3.14" upper="3.14" velocity="100" effort="100"/>
```

```
</joint>
```

- When choosing revolute, we first need to define which axis will be rotating.
- As we chose z (if you look at Fig, we want to rotate around the blue axis), we write "0 0 1", which means: no rotation on x and y, and a rotation on z.
- We set the revolution between -180 and +180 degrees (about -3.14 and 3.14 radians). We also have to specify a value for the velocity and effort limits. These two will usually be overridden by other ROS nodes. Set them to 100 by default; it won't be important here.



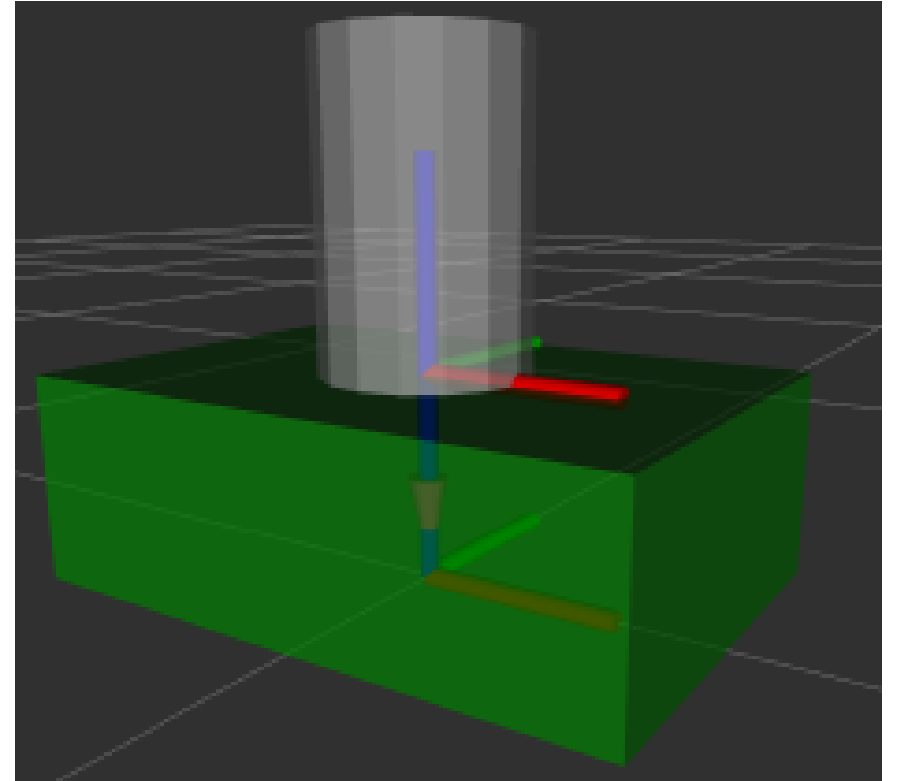
- With this joint, you have created a TF that defines the position of the `shoulder_link` relative to the `base_link`, and the movement between those two links.
- Now, you can start RViz again (disable the visual), and you will find a cursor in the **Joint State Publisher** window.
- Move the cursor to make the `shoulder_link` rotate on top of the `base_link`. Once again, it's all about making the frame correctly move.
- If the visual still doesn't look right, don't worry about it; this is what we will fix in the last step of this process.

- We can now fix the origin of the `shoulder_link` visual. Don't modify the `base_link` origin, as it's already correct. Here, we only modify the visual for the child link.
- To do so, you can enable the `shoulder_link` visual again on RViz and see that the frame is at the center of the cylinder.
- Thus, we need to offset the visual by half of the length of the cylinder, which means 0.15 m (half of 0.3 m).

- Modify the <origin> tag inside the <link> tag of the shoulder_link:

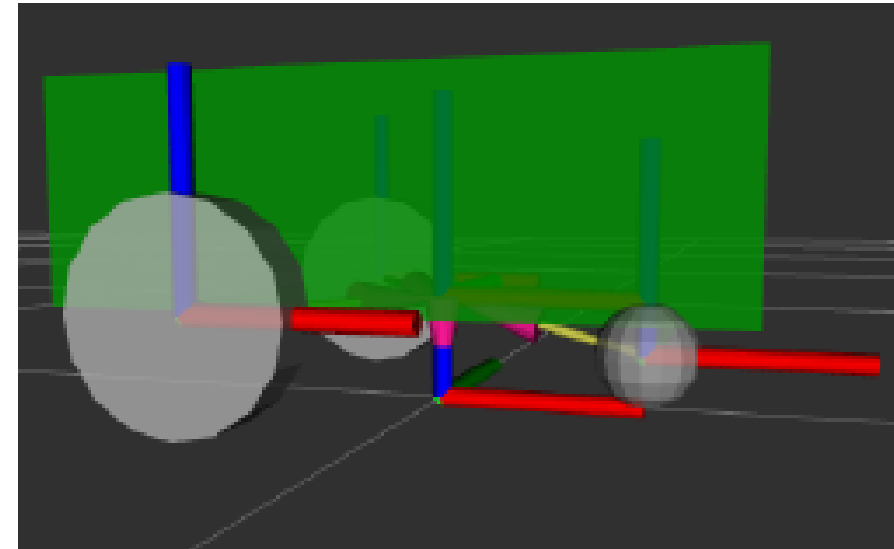
```
<origin xyz="0 0 0.15" rpy="0 0 0" />
```

- If you start RViz again, you will then see that everything is correctly placed.
- The process is now finished

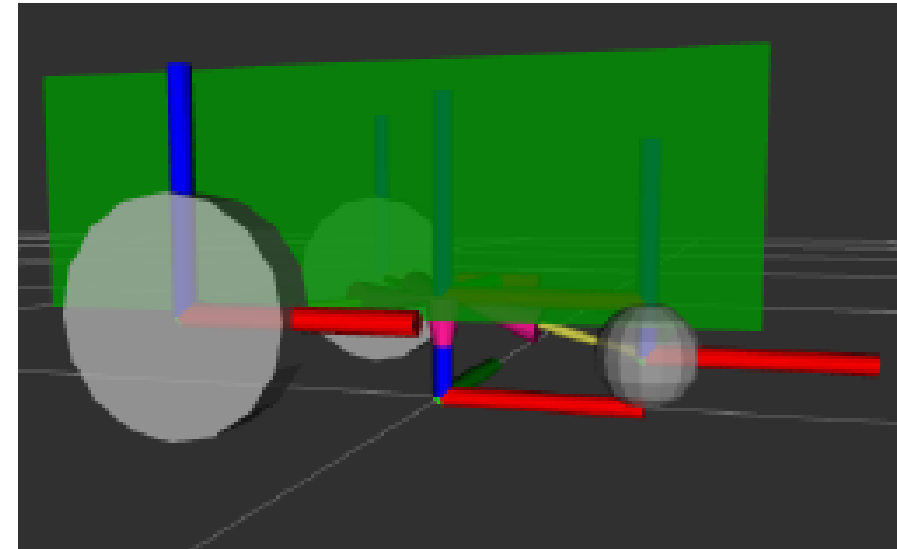


- Workspace
- Visualizing a Robot in RViz
- Creating a URDF
- **URDF for a Mobile Robot**

- Before writing any code, it's important to define what we want to achieve. The final result will look like this in Rviz.
- We will start the URDF with the robot's main structure (chassis), represented by a box. This box is the same as the one we have created with the `base_link`.
- Then, we will add two wheels, on each side of the chassis. Those two wheels will have a continuous rotation.
- Finally, for stability, we add a caster wheel (sphere) that will help the robot not fall forward when we simulate it in Gazebo. This caster wheel will be a fixed joint, and we won't add any movement to it.



- On top of that, at the end, we will also add another link named **base_footprint** (with no visual, we can consider it as a virtual link), which is the projection of the base_link on the ground.
- To get started, go back to the my_robot.urdf file we have written previously, and keep the base_link. Remove the shoulder_link as well as the base_shoulder_joint.
- Let's now add the two wheels to the side of the robot.



- To add the right wheel, we will follow the five-step process we just described.
- Let's name this link **right_wheel_link**.
- For the visual, we will use a cylinder, with a radius of 0.1 m and a length of 0.05 m.

- Here is the code for the link (*Step 1*):

```
<link name="right_wheel_link">
```

```
  <visual>
```

```
    <geometry>
```

```
      <cylinder radius="0.1" length="0.05" />
```

```
    </geometry>
```

```
    <origin xyz="0 0 0" rpy="0 0 0" />
```

```
    <material name="gray" />
```

```
  </visual>
```

```
</link>
```

- As the wheel will be attached to the base_link (green color), we choose the gray color to make a contrast. As you can see, we set all origin elements to 0. We will only come back to those elements at the end of the process.
- Now, let's add a joint between the base and the right wheel (*Step 2*).

```
<joint name="base_right_wheel_joint" type="fixed">  
  <parent link="base_link" />  
  <child link="right_wheel_link" />  
  <origin xyz="0 0 0" rpy="0 0 0" />  
</joint>
```

- For now, we set the type to **fixed**, so we can set the joint origin first, and then specify the type of movement. This will make things simpler and less prone to errors.
- With this code, you can already visualize the URDF in RViz. Disable the right_wheel_link visual (RobotModel | Links) as we don't need this for now, and it could make us confused when setting the joint origin.

- Then, the question is: where do we place the `right_wheel_link` frame relative to the `base_link` frame? Let's see for each axis:
- **X translation (red axis):** We want the wheel to be a bit behind, let's choose a -0.15 m offset.
- **Y translation (green axis):** The wheel should be on the side of the robot. Here, you have two choices (to understand this, make sure to have RViz open with the URDF).

- You can place the wheel origin just on the right side of the box. As the Y -axis is pointing left, the offset would be -0.2m (half of the width of the box).
- Then, later on, you would need to add an offset in the wheel visual, like we did for the `shoulder_link` in the previous example.
- Alternatively, you can add a small additional offset to the joint, so that the wheel would be outside of the box, and the visual would be centered around the frame.

- This second option is a good idea for wheels and some sensors—for example, when using a lidar, this is required to make the scan work properly. We then need to add -0.2 m, and an additional -0.025 m (half of the length of the wheel). The total offset is -0.225 m.
- **Z translation (blue axis):** There is no need for any offset here, as we want the center of the wheel to be at the bottom of the box.

- We have our three values for the translation. That's all we need. You might think that we have to rotate the joint axis because the visual is not correctly orientated.
- However, this is one of the most common mistakes, and it's where you could start to modify the wrong values to fix the wrong problem.
- As we saw in the process, we first fix the joint with the visual disabled, and then, and only then, we fix the visual.

- Let's modify the <origin> tag inside the <joint> tag (again: not in the link, only in the joint):

```
<origin xyz="-0.15 -0.225 0" rpy="0 0 0" />
```

- Then, start RViz again, disable the visual for the wheel (this visual is still wrong, but not a problem), and you can see that the joint is placed a bit outside the box.
- We can now easily add the movement (*Step 4*). As the wheel will continuously rotate (there is no minimum or maximum position), we choose the continuous type.

- We also need to specify the rotation axis. By looking at the robot model in RViz, we can see that we have to pick the Y -axis (the wheel should rotate around the green axis).
- Thus, we modify the `<joint>` tag accordingly:

```
<joint name="base_right_wheel_joint" type="continuous">  
  <parent link="base_link" />  
  <child link="right_wheel_link" />  
  <origin xyz="-0.15 -0.225 0" rpy="0 0 0" />  
  <axis xyz="0 1 0" />  
</joint>
```

- There is no need to specify a `<limit>` tag for continuous, as we did with the revolute type previously.
- You can now start RViz again, disable the wheel visual, and move the new cursor named `base_right_wheel_link` on the **Joint State Publisher** window.
- You should see the joint correctly rotating around the base.

- We can now finish the process and fix the visual for the link (*Step 5*). If you re-enable the wheel visual, you will see that it's not correctly orientated.
- You would need to add a 90-degree rotation on the X -axis (around the red axis). This corresponds to $\pi/2$, or about 1.57 radian (we will see later in this chapter how to use a precise value of π).
- Let's modify the `<origin>` tag for the `right_wheel_link`:
`<origin xyz="0 0 0" rpy="1.57 0 0" />`

- Start RViz again, and now everything should be fine: the wheel visual will be correctly placed (just outside of the box) and with the right orientation.
- When you move the cursor for the joint, the wheel will turn correctly.
- The right wheel was probably the most complicated part of this robot, but as you can see, if you follow the process in the order, there should be no problem. You can be sure that all values are correct, and that there is no error that will propagate onto the next links that you add.

- As the left wheel is the same as the right wheel, but on the opposite side of the box, writing the code will be fairly straightforward. Here I will just show you the final `<link>` and `<joint>` tags.
- Let's start with the link:

```

<link name="left_wheel_link">
  <visual>
    <geometry>
      <cylinder radius="0.1" length="0.05" />
    </geometry>
    <origin xyz="0 0 0" rpy="1.57 0 0" />
    <material name="gray" />
  </visual>
</link>

```

- This is the same code as for the right_wheel_link.
- Then, the left wheel will be connected to the chassis, so the parent of the left_wheel_link will be base_link.
- Here is the code for the joint:

```
<joint name="base_left_wheel_joint" type="continuous">  
  <parent link="base_link" />  
  <child link="left_wheel_link" />  
  <origin xyz="-0.15 0.225 0" rpy="0 0 0" />  
  <axis xyz="0 1 0" />  
</joint>
```

- Everything is the same, except for the offset on the Y-axis. For the right wheel, we had to go on the negative side, but for the left wheel, we went on the positive side.
- The important thing to check is that when you move the two cursors on the **Joint State Publisher** window on the positive side, both wheels are rotating in the same direction. If you have this, then your differential drive system is correctly designed.
- We can now add the **caster wheel** for the stability of the robot.

- When we add physics and gravity later on in the Gazebo simulator, you can guess that the robot would fall on the front side, because it's out of balance.
- To fix this, we will add what's called a caster wheel. It's the same principle as the wheels under a desk chair.
- To make things simple, the caster wheel will be represented by a sphere (with a radius of 0.05 m). For the movement, even if the wheel is rotating, this is not a rotation we control with ROS. It is a passive rotation; thus, we will consider the joint as fixed.

- Let's create the link first (Step 1):

```
<link name="caster_wheel_link">
```

```
<visual>
```

```
<geometry>
```

```
<sphere radius="0.05" />
```

```
</geometry>
```

```
<origin xyz="0 0 0" rpy="0 0 0" />
```

```
<material name="gray" />
```

```
</visual>
```

```
</link>
```

- There's nothing too complicated here. Now, the caster wheel will be connected to the chassis of the robot. Let's add the joint (*Step 2*). To follow the process, we first set all origin elements to 0:

```
<joint name="base_caster_wheel_joint" type="fixed">
  <parent link="base_link" />
  <child link="caster_wheel_link" />
  <origin xyz="0 0 0" rpy="0 0 0" />
</joint>
```

- Start RViz and disable the visual. From this, let's see where to place the origin of the caster wheel relative to the origin of the base (*Step 3*):
- **X translation:** As the two wheels are on the back of the robot, the caster wheel should be at the front. We can choose for example 0.2 m.
- **Y translation:** For better stability, we want the caster wheel to be centered, so, 0.
- **Z translation:** I specified 0.05 as the radius so that the diameter of the caster wheel (0.1 m) corresponds to the radius of the wheel. Thus, in order for the wheels and caster wheels to be aligned on the ground, we need to offset the Z-axis by -0.05 m. If you are not so sure about this, it's simple: try some values and see the results in RViz.

- There is no need to set any rotation for the joint, as we won't move the caster wheel, and it's a sphere.
- Let's apply the offset in the <origin> tag of the <joint> tag:

```
<origin xyz="0.2 0 -0.05" rpy="0 0 0" />
```

- Now the caster wheel will be correctly placed under the chassis, and you can verify that the bottoms of both wheels and caster wheel seem to be aligned. Here, there is no need to set a movement type (*Step 4*) nor to fix the visual (*Step 5*).

- The robot is correctly designed and will work fine when we add control to it. However, there is one improvement we can make.
- If you look at the robot in RViz, the base_link frame is not at the same altitude (z offset) as the bottom of the three wheels.
- This is fine, but it would be nice to have the origin of the robot aligned with the ground where the robot will be

- Not only will it make the robot look better in RViz (although it's not important), but it will also make things easier in the future.
- An example would be when you want to create a transformation from a docking station to a mobile robot, or from one robot to another one.
- Another example would be if you want to use the **Navigation 2** stack. If all robot origins are on the ground, you can then work in 2D, which is easier to handle.

- For that reason, it's quite common to add a virtual link named `base_footprint`, which will be the projection of the `base_link` on the ground.
- We say the link is *virtual* because it doesn't contain any visuals; it's just an additional frame we define in the space.
- Here is the code for the link:

```
<link name="base_footprint" />
```

- As you can see, this link is extremely simple, as we didn't include any `<visual>` tag.
- For a link name, we usually start with the rigid part name and add the `_link` suffix. Here, we make an exception.
- You will find this `base_footprint` name in many URDF files for existing mobile robots.

- Now, we can add a new fixed joint, with the base_footprint as the parent, and the base_link as the child:

```
<joint name="base_joint" type="fixed">  
  <parent link="base_footprint" />  
  <child link="base_link" />  
  <origin xyz="0 0 0.1" rpy="0 0 0" />  
</joint>
```

- We apply a 0.1 m offset on the Z-axis, which corresponds to the right and left wheels' radius.

Thank you for your attention.

Any Questions?

